Tema 2: Estructuras de control

Fundamentos de Programación

Francisco J. Rodríguez Díaz
(fjrodriguez@decsai.ugr.es)
Departamento de Ciencias de la Computación e I.A.

Universidad de Granada

Tabla de Contenidos I

1. Estructuras condicionales
2. Estructuras repetitivas
3. Tipo de dato enumerado

Flujo de control

Flujo de control

- Es la especificación del orden de ejecución de las sentencias.
- Por defecto es secuencial, se ejecutan sucesivamente según el orden de aparición.

Código

```
1  double a, b, c, denominador, radical, raiz1, raiz2;
2
3  cout <<"Introduce coeficiente de 2º grado:";
4  cin >> a;
5  cout <<"Introduce coeficiente de 1er grado:";
6  cin >> b;
7  cout <<"Introduce coeficiente independiente:";
8  cin >> c;
9  denominador = 2*a;
10 radical = sqrt(b*b - 4*a*c);
11 raiz1 = (-b + radical)/denominador;
12 raiz2 = (-b - radical)/denominador;
13 cout <<"Las raíces son " <<raiz1 <<" y " <<raiz2 <<endl;
```

Flujo de control

Flujo de control

- Es la especificación del orden de ejecución de las sentencias.
- Por defecto es secuencial, se ejecutan sucesivamente según el orden de aparición.

Código

```
1  double a, b, c, denominador, radical, raiz1, raiz2;
2
3  cout <<"Introduce coeficiente de 2º grado:";
4  cin >> a;
5  cout <<"Introduce coeficiente de 1er grado:";
6  cin >> b;
7  cout <<"Introduce coeficiente independiente:";
8  cin >> c;
9  denominador = 2*a;
10 radical = sqrt(b*b - 4*a*c);
11 raiz1 = (-b + radical)/denominador;
12 raiz2 = (-b - radical)/denominador;
13 cout <<"Las raíces son " <<raiz1 <<" y " <<raiz2 <<endl;
```

Flujo: 3-12

Modificaciones de Flujo

Se puede modificar el flujo

- **Condiciones**: Ejecutar una serie de instrucciones o no según una condición.
- **Bucles**: Ejecutar una serie de instrucciones varias veces.

Ejemplo: factorial(n)

- Condiciones: Devolver 1 si el valor es cero.
- Bucle: Multiplicar los valores de forma repetida, supone hacer n multiplicaciones.

$$factorial(n) \Rightarrow \prod_{i=1}^{n} i = 1 * 2 * ... * n.$$

Contenido del tema

1. Estructuras condicionales
2. Estructuras repetitivas
3. Tipo de dato enumerado

Estructura condicional simple

Formato condición simple

```
if (<condition>)
   <bloque_if>
```

Formato

- **condition:** es una condición lógica.
- **bloque_if:** es una instrucción (o conjunto) que se ejecuta si la condición es cierta.

Ejemplo

```
if (x < 0)
   cout <<"x es negativo" <<endl;

if (x >= 0) {
   cout <<"x es positivo o cero" <<endl;

   if (x == 0) {
      cout <<"x es cero" <<endl;
   }
}
```

Diagrama de flujo

Diagrama de flujo

Para representar el flujo de un programa se usan diagramas de flujo.

- Un bloque representa una instrucción (o grupo).
- Un rombo representa una condición.
- Las flechas indican el flujo de ejecución.

Código

```
1  if (x < 0) {
2     cout <<"x es negativo";
3  }
4
5  cout <<"Termino";
```

Diagrama de flujo

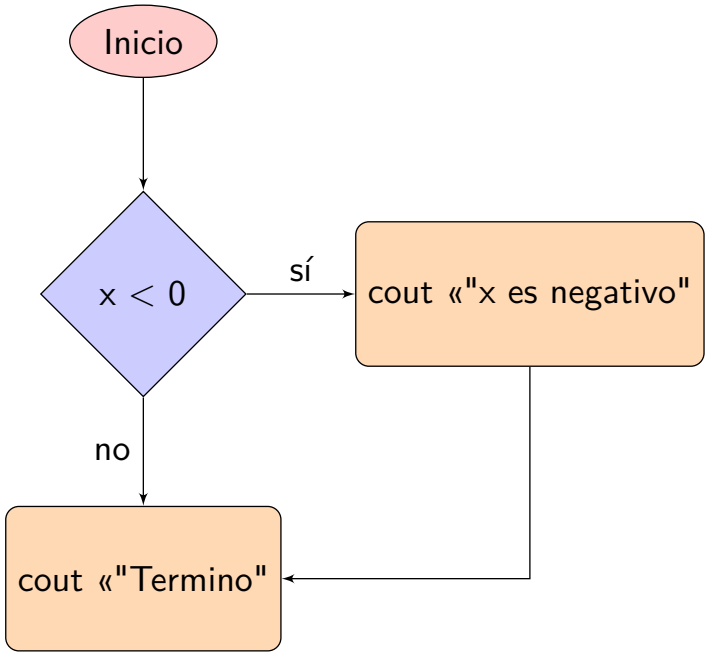
Diagrama de flujo

Para representar el flujo de un programa se usan diagramas de flujo.

- Un bloque representa una instrucción (o grupo).
- Un rombo representa una condición.
- Las flechas indican el flujo de ejecución.

Código

```
1  if (x < 0) {
2     cout <<"x es negativo";
3  }
4
5  cout <<"Termino";
```



Sentencias condicionales

Bloques

Si hay varias sentencias, es necesario encerrarlas entre llaves.

Sin llaves

```
x = 5;

if (x < 0)
   cout <<"x es negativo";
   x *= -1;

cout <<"x vale: " <<x <<endl;
```

Sentencias condicionales

Bloques

Si hay varias sentencias, es necesario encerrarlas entre llaves.

Sin llaves

```
1  x = 5;
2
3  if (x < 0)
4     cout <<"x es negativo";
5     x *= -1;
6
7  cout <<"x vale: " <<x <<endl;
```

x vale -5

Sentencias condicionales

Bloques

Si hay varias sentencias, es necesario encerrarlas entre llaves.

Sin llaves

```
1  x = 5;
2
3  if (x < 0)
4     cout <<"x es negativo";
5     x *= -1;
6
7  cout <<"x vale: " <<x <<endl;
```

x vale -5

Con llaves

```
1  x = 5
2
3  if (x < 0) {
4     cout <<"x es negativo";
5     x *= -1
6  }
7
8  cout <<"x vale: " <<x <<endl;
```

Sentencias condicionales

Bloques

Si hay varias sentencias, es necesario encerrarlas entre llaves.

Sin llaves

```
1  x = 5;
2
3  if (x < 0)
4     cout <<"x es negativo";
5     x *= -1;
6
7  cout <<"x vale: " <<x <<endl;
```

x vale -5

Con llaves

```
1  x = 5
2
3  if (x < 0) {
4     cout <<"x es negativo";
5     x *= -1
6  }
7
8  cout <<"x vale: " <<x <<endl;
```

x vale 5

Sentencias condicionales

Bloques

Si hay varias sentencias, es necesario encerrarlas entre llaves.

Sin llaves

```
1  x = 5;
2
3  if (x < 0)
4     cout <<"x es negativo";
5     x *= -1;
6
7  cout <<"x vale: " <<x <<endl;
```

x vale -5

Con llaves

```
1  x = 5
2
3  if (x < 0) {
4     cout <<"x es negativo";
5     x *= -1
6  }
7
8  cout <<"x vale: " <<x <<endl;
```

x vale 5

Recomiendo usar llaves **siempre**.

No equivocarse en la condición

Cuidado

```
1  int valor1 = 4;
2  int valor2 = 3;
3
4  if (valor1 = valor2) {
5     cout <<"Son iguales" <<endl;
6  }
```

No equivocarse en la condición

Cuidado

```
1  int valor1 = 4;
2  int valor2 = 3;
3
4  if (valor1 = valor2) {
5     cout <<"Son iguales" <<endl;
6  }
```

¿Qué salida da?

No es una comparación, es una asignación.

- Devuelve el valor asignado.
- Si es cero considera que no es cierto, no ejecuta bloque.
- Si no es cero considera que es cierto, ejecuta el bloque

No equivocarse en la condición

Cuidado

```
int valor1 = 4;
int valor2 = 3;

if (valor1 = valor2) {
   cout <<"Son iguales" <<endl;
}
```

No es una comparación, es una asignación.

- Devuelve el valor asignado.
- Si es cero considera que no es cierto, no ejecuta bloque.
- Si no es cero considera que es cierto, ejecuta el bloque

Arreglado

```
int valor1 = 4;
int valor2 = 3;

if (valor1 == valor2) {
   cout <<"Son iguales" <<endl;
}
```

Algún ejemplo más

Ejemplo

```
1  int entero;
2
3  cin >>entero;
4
5  if (entero % 2 == 0) {
6     cout <<endl <<"Es par" <<endl;
7  }
8
9  cout <<"Fin programa";
```

Estructuras condicionales animadas

¿Qué ocurre si hay dos estructuras condicionales simples?

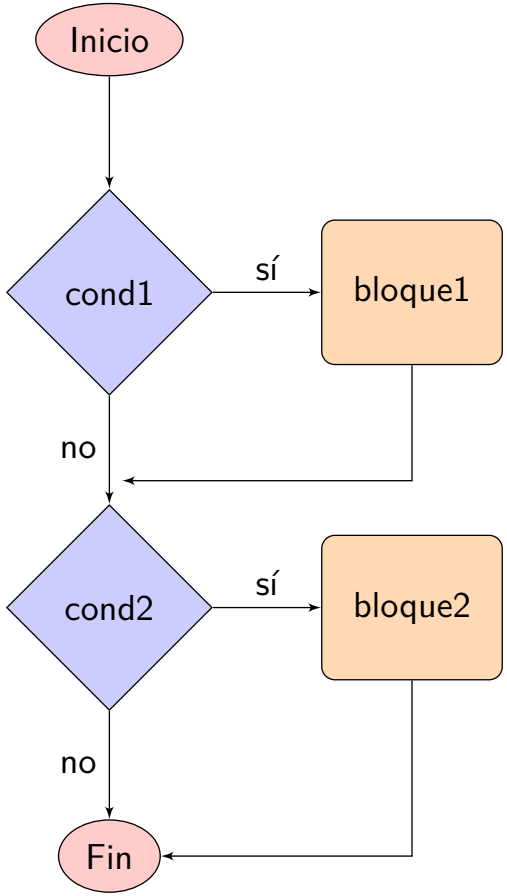
```
1  if (cond1)
2     bloque1
3
4  if (cond2)
5     bloque2
```

Estructuras condicionales animadas

¿Qué ocurre si hay dos estructuras condicionales simples?

```
1  if (cond1)
2     bloque1
3
4  if (cond2)
5     bloque2
```

- La ejecución de bloque1 depende sólo de cond1.
- La ejecución de bloque2 depende sólo de cond2.



Ejemplo de código con dos condiciones

Código

```
1  int entero;
2
3  cin >>entero;
4
5  if (entero % 2 == 0) {
6     cout <<endl <<"Es par" <<endl;
7  }
8
9  if (entero % 2 != 0) {
10    cout <<endl <<"Es impar" <<endl;
11 }
12
13 cout <<"Fin programa";
```

Ejemplo de código con dos condiciones

Código

```
1  int entero;
2
3  cin >>entero;
4
5  if (entero % 2 == 0) {
6     cout <<endl <<"Es par" <<endl;
7  }
8
9  if (entero % 2 != 0) {
10    cout <<endl <<"Es impar" <<endl;
11 }
12
13 cout <<"Fin programa";
```

- Ambas condiciones son contradictorias.
- Hay una forma más simple, usando **else**.

Sentencia condicional con else

Formato condición simple

```
1 if (<condition>)
2    <bloque_if>
3 else
4    <bloque_else>
```

Formato

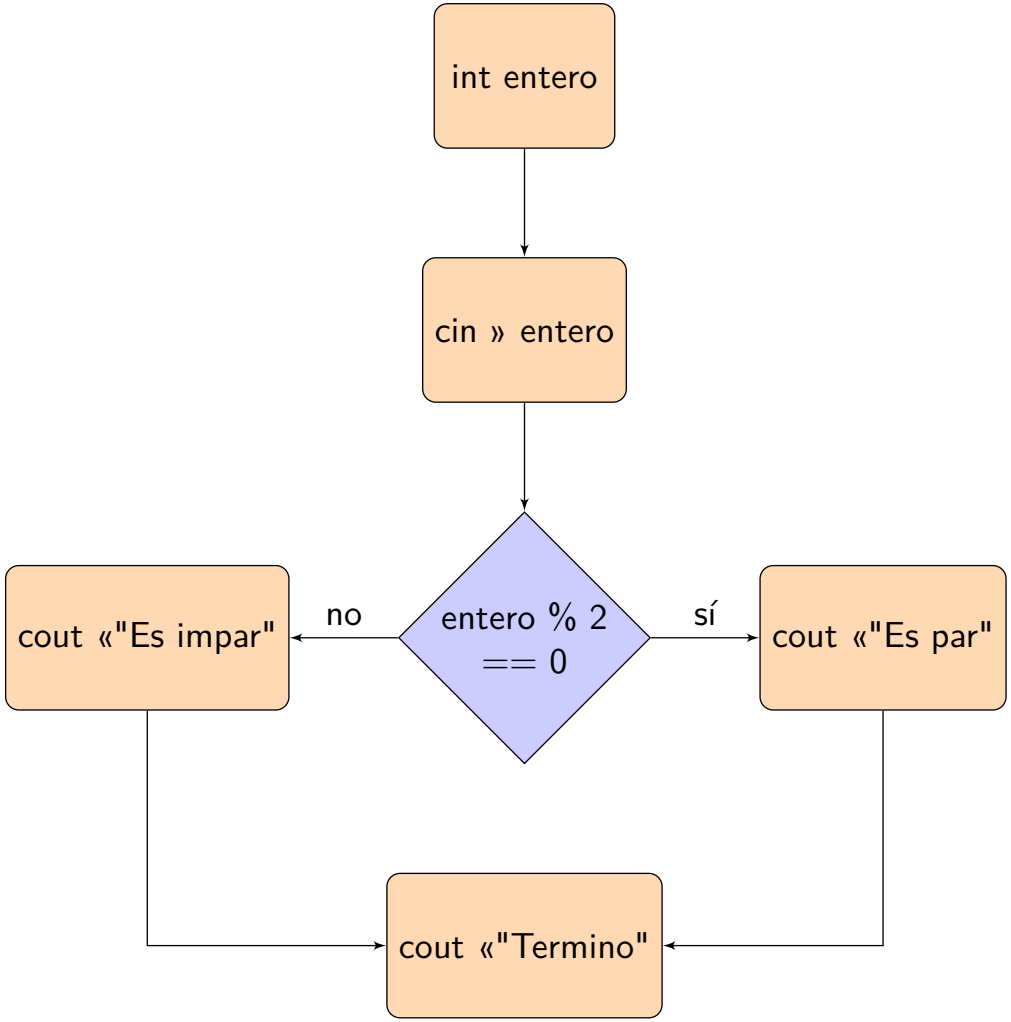
- **condition:** condición lógica.
- **bloque_if:** instrucción (o conjunto) que se ejecuta si la condición es cierta.
- **bloque_else:** instrucción (o conjunto) que se ejecuta si la condición es falsa.

Ejemplo de sentencia condicional con else

Ejemplo

```
1  int entero;
2
3  cin >>entero;
4
5  if (entero % 2 == 0) {
6     cout <<endl <<"Es par" <<endl;
7  }
8  else {
9     cout <<endl <<"Es impar" <<endl;
10 }
11
12 cout <<"Fin programa";
```

Flujo de una sentencia con else



Mayor legibilidad

Permiten reducir las condiciones, más legible.

Izda o Dcha

```
1   if (entero % 2 == 0)
2       ...
3   if (entero % 2 != 0)
4       ...
5
6   if (edad >= 18)
7       ...
8   if (edad < 18)
9       ...
```

```
1   if (entero % 2 == 0)
2       ...
3   else
4       ...
5
6   if (edad >= 18)
7       ...
8   else
9       ...
```

Mayor legibilidad

Permiten reducir las condiciones, más legible.

Izda o Dcha

```
1   if (entero % 2 == 0)
2       ...
3   if (entero % 2 != 0)
4       ...
5
6   if (edad >= 18)
7       ...
8   if (edad < 18)
9       ...
```

```
1   if (entero % 2 == 0)
2       ...
3   else
4       ...
5
6   if (edad >= 18)
7       ...
8   else
9       ...
```

Menos condiciones son más fáciles de leer.

Algunas condiciones complejas

Combinando operaciones

- Es común combinar distintas condiciones con && o ||.

Problema de subida de sueldo (ficticio)

En una empresa se desea subir el salario medio a quien gane menos de 1000 euros. La subida será del 4% a quien tenga 45 años o más, y del 1% a quien tenga menos. ¿Cómo sería el código?

Evitar expresiones complejas

Simplificar expresiones

- Algunas veces se pueden simplificar expresiones.

Reglas

- !A && !B $\leftrightarrow$!(A || B)
- !A || !B $\leftrightarrow$!(A && B)
- (A && B) || (A && C) $\leftrightarrow$ A && (B || C)
- (A || B) && (A || C) $\leftrightarrow$ A || (B && C)

Ejemplos

- !(edad $<$ 16) && (edad $<$ 40)
- ((joven && numero_hijos $>$ 2) || (joven && (numero_hijos $>$ 1) || dependiente))

Múltiples condiciones

Estructura if-else

- A menudo es muy común comprobar el valor de una variable.

Versión 1

```
1  char option;
2
3  cout <<"Menu: " <<endl;
4
5  cin >>option;
6
7  if (option == 'S') {
8     cout <<"Save file";
9  }
10
11 if (option == 'O') {
12    cout <<"Open file";
13 }
14
15 if (option == 'R') {
16    cout <<"Reload file";
17 }
```

Múltiples condiciones

Estructura if-else

- A menudo es muy común comprobar el valor de una variable.

Versión 1

```
1  char option;
2
3  cout <<"Menu: " <<endl;
4
5  cin >>option;
6
7  if (option == 'S') {
8     cout <<"Save file";
9  }
10
11 if (option == 'O') {
12    cout <<"Open file";
13 }
14
15 if (option == 'R') {
16    cout <<"Reload file";
17 }
```

¿Es lo más adecuado?

Múltiples condiciones

Estructura if-else

- A menudo es muy común comprobar el valor de una variable.

Versión 1

```
1  char option;
2
3  cout <<"Menu: " <<endl;
4
5  cin >>option;
6
7  if (option == 'S') {
8     cout <<"Save file";
9  }
10
11 if (option == 'O') {
12    cout <<"Open file";
13 }
14
15 if (option == 'R') {
16    cout <<"Reload file";
17 }
```

¿Es lo más adecuado?

¿Las opciones son compatibles?

Múltiples condiciones

Estructura if-else

- A menudo es muy común comprobar el valor de una variable.

Versión 2

```
1  char option;
2
3  cout <<"Menu: " <<endl;
4
5  cin >>option;
6
7  if (option == 'S') {
8     cout <<"Save file";
9  }
10 else if (option == 'O') {
11    cout <<"Open file";
12 }
13 else if (option == 'R') {
14    cout <<"Reload file";
15 }
16 else {
17    cout <<"Opción no conocida" <<endl;
```

Múltiples condiciones

Estructura if-else

- A menudo es muy común comprobar el valor de una variable.

Versión 2

```
1  char option;
2
3  cout <<"Menu: " <<endl;
4
5  cin >>option;
6
7  if (option == 'S') {
8     cout <<"Save file";
9  }
10 else if (option == 'O') {
11    cout <<"Open file";
12 }
13 else if (option == 'R') {
14    cout <<"Reload file";
15 }
16 else {
17    cout <<"Opción no conocida" <<endl;
```

Más legible, bastante conciso.

Múltiples condiciones

Estructura if-else-if

- Adecuado cuando las condiciones son mutuamente excluyentes.
- Muy común al comparar el valor de una variable varias veces.

Ventajas

- Más conciso.
- Más legible.
- Permite añadir una opción distinta final fácilmente.
- Muy general.

Sentencia switch

Sentencia switch

- Sentencia condicional, depende de los valores de una variable.
- Muy utilizado en menús.

sintaxis switch

```
switch (<variable>) {

   case valor1: <bloque_valor1>;
                break;

   case valor2: <bloque_valor2>;
                break;

   ...

   default: <sentencias_defecto>;
            break;
}
```

Sólo funciona con variable de tipo entero. Usar if-else-if para otro tipo.

Código original

Versión if-else-if

```
1  char option;
2
3  cout <<"Menu: " <<endl;
4
5  cin >>option;
6
7  if (option == 'S') {
8     cout <<"Save file";
9  }
10 else if (option == 'O') {
11    cout <<"Open file";
12 }
13 else if (option == 'R') {
14    cout <<"Reload file";
15 }
16 else {
17    cout <<"Opción no conocida" <<endl;
18 }
```

Sentencia switch

Versión switch

```
1   char opcion;
2
3   cin >>opcion;
4
5   switch (opcion) {
6      case 'S': cout <<"Save file";
7                break;
8
9      case 'O': cout <<"Open file";
10               break;
11
12     case 'R': cout <<"Relod file";
13               break;
14
15     default: cout <<"Opción no conocida" <<endl;
16              break;
17  }
```

Sentencia switch

Versión switch

```
1   char opcion;
2
3   cin >>opcion;
4
5   switch (opcion) {
6      case 'S': cout <<"Save file";
7                break;
8
9      case 'O': cout <<"Open file";
10               break;
11
12     case 'R': cout <<"Relod file";
13               break;
14
15     default: cout <<"Opción no conocida" <<endl;
16              break;
17  }
```

¿Por qué necesita *break*?

Sentencia switch

Versión switch

```
1  char opcion;
2
3  cin >> opcion;
4
5  switch (opcion) {
6     case 's':
7     case 'S': cout <<"Save file";
8               break;
9
10    case 'o':
11    case 'O': cout <<"Open file";
12              break;
13
14    case 'r':
15    case 'R': cout <<"Relod file";
16              break;
17
18    default: cout <<"Opción no conocida" <<endl;
19             break;
20 }
```

Sentencia switch

Versión switch

```
1  char opcion;
2
3  cin >> opcion;
4
5  switch (opcion) {
6     case 's':
7     case 'S': cout <<"Save file";
8               break;
9
10    case 'o':
11    case 'O': cout <<"Open file";
12              break;
13
14    case 'r':
15    case 'R': cout <<"Relod file";
16              break;
17
18    default: cout <<"Opción no conocida" <<endl;
19             break;
20 }
```

Permite agrupar argumentos

Sentencias if anidadas

Formato condición if-else anidada

```
if (<condition_1>) {
   if (<condicion_2>) {
      <bloque_if_2>
   }
   else {
      <bloque_else_2>
   }
}
else {
   <bloque_else_1>
}
```

Sentencias if anidadas

Formato condición if-else anidada

```
if (<condition_1>) {
   if (<condicion_2>) {
      <bloque_if_2>
   }
   else {
      <bloque_else_2>
   }
}
else {
   <bloque_else_1>
}
```

Limite recomendado

- Máximo 3 niveles

Sentencias if anidadas

Condición compuesta

```
1  if (<condition_1> && <condicion_2>){
2     <bloqueA>
3  }
4  else {
5     <bloqueB>
6  }
```

Condición anidada

```
1  if (<condition_1>) {
2     if (<condicion_2>) {
3        <bloqueA>
4     }
5     else {
6        <bloqueB>
7     }
8  }
9  else {
10    <bloqueB>
11 }
```

Sentencias if anidadas

Ejemplo

- Aplicar subida 4% cuando la edad del trabajador sea mayor o igual de 45 y su salario esté por debajo de 1300
- En otro caso 1%.

Subida condición compuesta

```
1 if (edad >= 45 && salario_base < 1300){
2    salario = salario * 1.04;
3 }
4 else {
5    salario = salario * 1.01;
6 }
```

Sentencias if anidadas

Subida condición anidada

```
if (edad >= 45) {
   if (salario_base < 1300) {
      salario = salario * 1.04;
   }
   else {
      salario = salario * 1.01;
   }
}
else {
   salario = salario * 1.01;
}
```

Sentencias if anidadas

Ejemplo

- Añadimos un 2% si además el número de hijos es mayor que 2
- La subida solo se aplica si la experiencia es mayor a 2 años
- En otro caso se baja un 1% el salario

Subida condición anidada

```
1  if (experienia > 2) {
2     if (edad >= 45 && salario_base < 1300) {
3        salario = salario * 1.04;
4        if (numero_hijos > 2) {
5           salario = salario * 1.02;
6        }
7     }
8     else {
9        salario = salario * 1.01;
10    }
11 }
12 else {
13    salario = salario * 0.99;
14 }
```

Sentencias if anidadas

Subida condición compuesta

```
1  if (experiencia > 2 && edad >= 45 && salario_base < 1300)
2     salario = salario * 1.04;
3  if (experiencia > 2 && edad >= 45 && salario_base < 1300 && num_hijos > 2)
4     salario = salario * 1.02;
5  if (experiencia > 2 && (edad < 45 || salario_base >= 1300))
6     salario = salario * 1.01;
7  if (experiencia <= 2)
8     salario = salario * 0.99;
```

Contenido del tema

1. Estructuras condicionales
2. Estructuras repetitivas
3. Tipo de dato enumerado

Definición y Tipos

Definición

Las estructuras repetitivas o bucles se usan para **ejecutar varias veces** un bloque de sentencias

Tipos (C++)

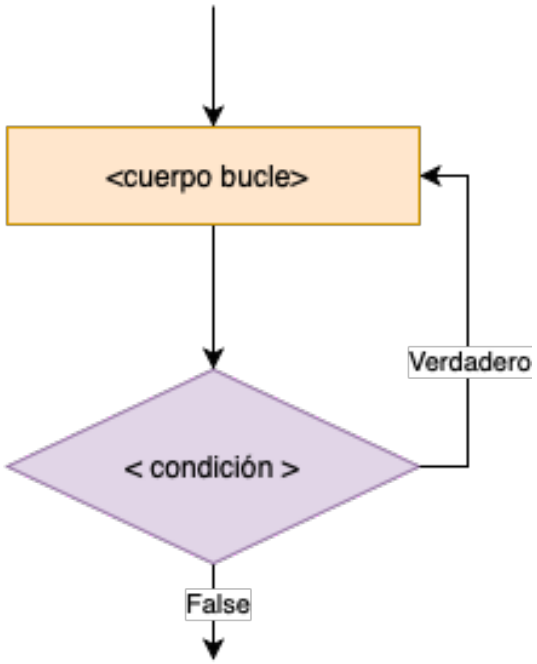
- do .. while
- while
- for

Do .. while

Bucle do .. while

- El bucle **do .. while** ejecuta un conjunto de sentencias mientras se cumpla una condición.
- Estas sentencias se ejecutan **siempre al menos una vez**.

```
1  ...
2  do {
3    <cuerpo_bucle>
4  } while (<condicion>);
5  ...
```

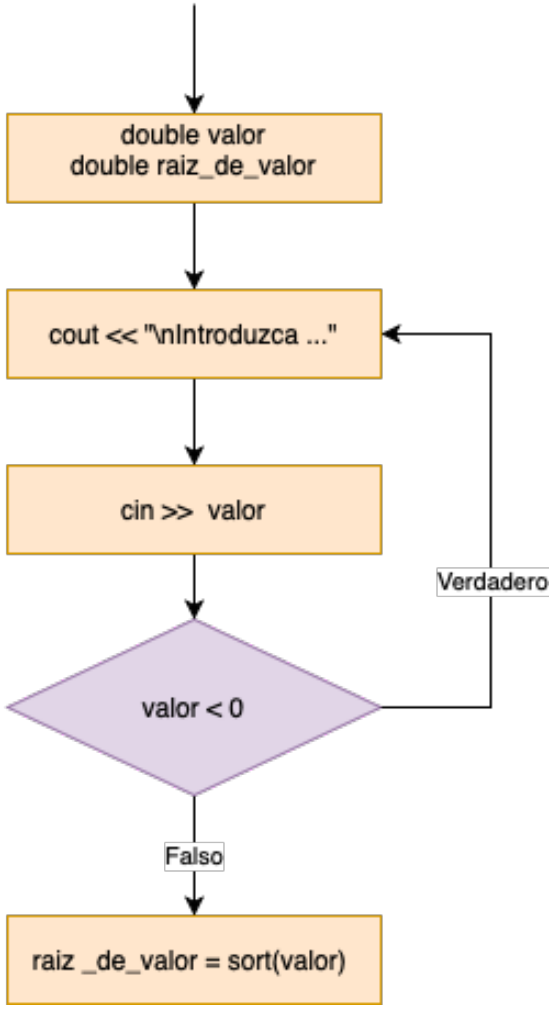


Do .. while: caso de uso

Entrada de Datos: filtro

```
1   int main() {
2     double valor;
3     double raiz de valor;
4
5     do {
6       cout << "\nIntroduzca un valor positivo: ";
7       cin >> valor;
8     } while (valor < 0);
9
10    raiz_de_valor = sqrt(valor);
11
12  }
```

Do .. while: caso de uso



Do .. while

Versión switch

```
1  int tope;
2  int num_lineas;
3
4  cout << "\n¿Cuántas líneas quieres imprimir?";
5
6  do {
7    cin >> tope;
8  } while (tope < 0);
9
10 num_lineas = 0;
11
12 do {
13   cout << '\n' << "*****";
14   num_lineas++;
15 } while (num_lineas < tope);
```

Do .. while

Versión switch

```
1  int tope;
2  int num_lineas;
3
4  cout << "\n¿Cuántas líneas quieres imprimir?";
5
6  do {
7    cin >> tope;
8  } while (tope < 0);
9
10 num_lineas = 0;
11
12 do {
13   cout << '\n' << "*****";
14   num_lineas++;
15 } while (num_lineas < tope);
```

¿Qué pasa cuando tope es 0?

Do .. while

Para diseñar un bucle while:

- ¿El cuerpo del bucle se debe ejecutar al menos una vez?
- Revisar cuidadosamente la **condición de salida**.
- Si se utiliza una variable iterador, revisar valor inicial, final y **su actualización**.

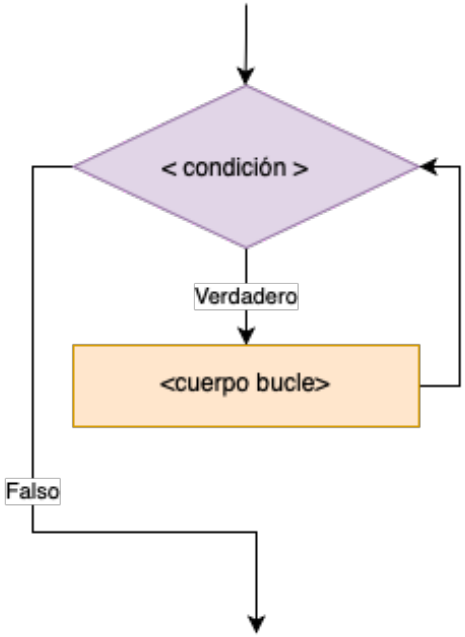
While

Bucle while

La principal **diferencia** con respecto a do .. while es que el **cuerpo del bucle puede no ejecutarse nunca** si no se cumple la condición del bucle.

While

```
1  ...
2  while (condicion) {
3    <cuerpo_bucle>
4  }
5  ...
```



While

Imprimir Líneas

```
1  int tope;
2  int num_lineas;
3
4  cout << "\n¿Cuántas líneas quieres imprimir?";
5
6  do {
7    cin << tope;
8  } while (tope < 0);
9
10 num_lineas = 0;
11
12 while (num_lineas < tope) {
13   cout << '\n' << "*****";
14   num_lineas++;
15 }
```

While

Sumar primeros 100 números

```
1  int suma, valor;
2
3  valor = 1;
4  suma = 0;
5
6  while (valor <= 100) {
7    suma = suma +  valor;
8    valor++;
9  }
```

Lectura de Datos

Lectura de datos

Leer datos por teclado, hasta que se introduzca un número negativo y obtener la suma de todos los números.

Do . . . While

```
1  int suma, valor;
2
3  suma = 0;
4
5  do {
6    cout << "Introduce un número mayor o igual que 0" << endl;
7    cin >> valor;
8    suma += valor;
9
10 } while (valor >= 0)
```

Lectura de Datos

Lectura de datos

Leer datos por teclado, hasta que se introduzca un número negativo y obtener la suma de todos los números.

Do . . . While

```
1  int suma, valor;
2
3  suma = 0;
4
5  do {
6    cout << "Introduce un número mayor o igual que 0" << endl;
7    cin >> valor;
8    suma += valor;
9
10 } while (valor >= 0)
```

¡Error! Suma también el último valor introducido.

Lectura de Datos

Do ... While

```
1  int suma, valor;
2
3  suma = 0;
4
5  do {
6    cout << "Introduce un número mayor o igual que 0" << endl;
7    cin >> valor;
8    if (valor >= 0) {
9      suma += valor;
10   }
11 } while (valor >= 0)
```

Lectura de Datos

Do ... While

```
1  int suma, valor;
2
3  suma = 0;
4
5  do {
6    cout << "Introduce un número mayor o igual que 0" << endl;
7    cin >> valor;
8    if (valor >= 0) {
9      suma += valor;
10   }
11 } while (valor >= 0)
```

Hay que evaluar dos veces la misma condición!

Lectura de Datos

Solución: lectura anticipada

```
1  int suma, valor;
2
3  suma = 0;
4
5  cout << "Introduce un número mayor o igual que 0" << endl;
6  cin >> valor;
7
8  while (valor >= 0) {
9    suma += valor;
10   cout << "Introduce un número mayor o igual que 0" << endl;
11   cin >> valor; //no olvidar esto, bucle infinito!
12 }
```

While

Algoritmo: Número de dígitos de un número

- Problema: 57102 -> 5 dígitos
- Algoritmo:
  - Entradas: n
  - Salidas: num_digitos
  - Descripción:
    1. Ir dividiendo n por 10 hasta llegar a una sola cifra (0-9).
    2. El número de dígitos será el número de iteraciones

While

Algoritmo: Número de dígitos de un número

- Problema: 57102 -> 5 dígitos
- Algoritmo:
  - Entradas: n
  - Salidas: num_digitos
  - Descripción:
    1. Ir dividiendo n por 10 hasta llegar a una sola cifra (0-9).
    2. El número de dígitos será el número de iteraciones

Número de dígitos de un número

```
cout << "\nIntroduce un número positivo: ";

cin >> n;
num_digitos = 1;

while (n > 9) {
  n = n / 10;
  num_digitos++;
}
```

Bucle sin fin

Bucles sin fin

```
contador = 1;

while (contador != 10) {
  contador = contador + 2;
}
```

Bucle sin fin

Bucles sin fin

```
contador = 1;

while (contador != 10) {
  contador = contador + 2;
}
```

Solución

```
contador = 1;

while (contador <= 10) {
  contador = contador + 2;
}
```

Condiciones Compuestas

1. Plantear las condiciones simples que la forman.
2. Pensar cuando queremos salir:
   - Todas son falsas simultáneamente -> conectamos con ||
   - Cualquiera de ellas es falsa -> conectamos con &&

Condiciones Compuestas

1. Plantear las condiciones simples que la forman.
2. Pensar cuando queremos salir:
   - Todas son falsas simultáneamente -> conectamos con ||
   - Cualquiera de ellas es falsa -> conectamos con &&

Ejemplo Menú

```
1  char opcion;
2
3  do {
4    cout << "¿Desea formatear el disco?:" << endl;
5    cin >> opcion;
6  } while (opcion != 's' opcion != 'S'
7          opcion != 'n' opcion!= 'N');
```

Condiciones Compuestas

Solución

```
1  char opcion;
2
3  do {
4    cout << "¿Desea formatear el disco?:" << endl;
5    cin >> opcion;
6  } while (opcion != 's' && opcion != 'S' &&
7          opcion != 'n' && opcion!= 'N');
```

While: caso de uso (saber si un número es primo)

Búsqueda: ¿es primo un número?

While: caso de uso (saber si un número es primo)

Búsqueda: ¿es primo un número?

```
1  int divisor, valor, ultimo_divisor_posible;
2  bool es_primo;
3
4  cin >> valor;
5
6  es_primo = true;
7  divisor = 2;
8  ultimo_divisor_posible = valor / 2;
9
10 while (divisor <= ultimo_divisor_posible && es_primo) {
11   if (valor % divisor == 0) {
12     es_primo = false;
13   }
14   else {
15     divisor++;
16   }
17 }
```

Bucles For

Motivación

- Para repetir un conjunto de sentencias un número de veces fijado de antemano.
- Se necesita una variable contadora, un valor inicial, un valor final y un incremento.

Bucles For

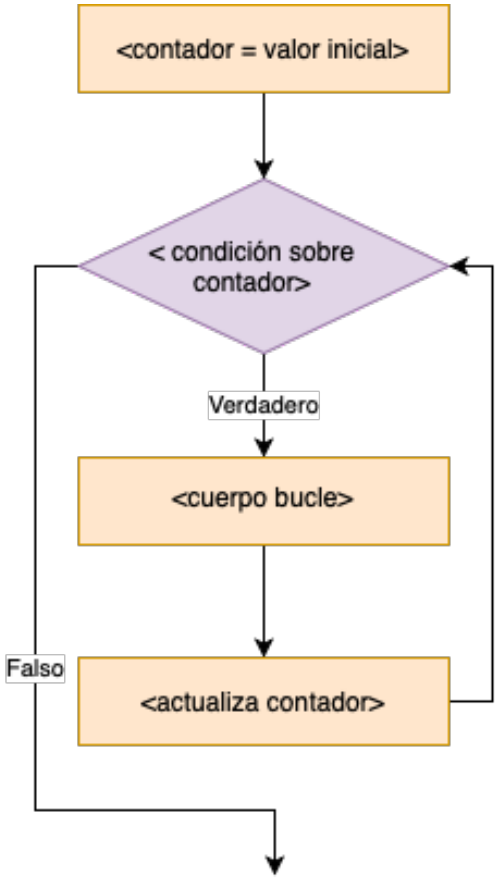
Media aritmética de 5 números

```
1  int main() {
2    int contador, valor, suma, tope;
3    double media;
4
5    tope = 5;
6    suma = 0;
7    contador = 1; //INICIALIZACIÓN
8
9    while (contador <= tope) { //COMPROBACIÓN VALOR FINAL
10     cout << "\nIntroduce un número: ";
11     cin >> valor;
12     suma = suma + valor;
13
14     contador = contador + 1; //INCREMENTO
15   }
16
17   media = suma / (tope * 1.0);
18   cout << "\nLa media es " << media;
19
20 }
```

Bucles For

Estructura For

```
1 for (<contador = valor inicial>; <condición contador>; <actualiza contador>) {
2   <cuerpo del bucle>
3 }
```



Bucles For

Media aritmética de 5 números

```
1  int main() {
2    int contador, valor, suma, tope;
3    double media;
4
5    tope = 5;
6    suma = 0;
7
8    for (contador = 1; contador <= tope; contador++)
9      cout << "\nIntroduce un número: ";
10     cin >> valor;
11     suma = suma + valor;
12   }
13
14   media = suma / (tope * 1.0);
15   cout << "\nLa media es " << media;
16 }
```

Anidamiento de Bucles

Tabla de Multiplicar

```
1  int main() {
2    const int TOPE_IZDA = 3;
3    const int TOPE_DCHA = 3;
4
5    int izda, dcha;
6
7    for (izda = 1; izda <= TOPE_IZDA; izda++) {
8      for (dcha = 1; dcha <= TOPE_DCHA; dcha++) {
9        cout << izda << " * " << dcha << " = " << izda * dcha << "\t";
10     }
11     cout << endl;
12   }
13 }
```

	dcha=1	dcha=2	dcha=3
izda=1	1*1 = 1	1*2 = 2	1*3 = 3
izda=2	2*1 = 2	2*2 = 4	2*3 = 6
izda=3	3*1 = 3	3*2 = 6	3*3 = 9

Anidamiento de Bucles

¿Qué salida produce este código?

```
1  int suma, iteraciones;
2
3  suma = iteraciones = 0;
4
5  for (int i = 1; i <= 5; i++) {
6    for (int j = 1; j <= 5; j++) {
7      suma = suma + j;
8      iteraciones ++;
9    }
10 }
```

Anidamiento de Bucles

¿Qué salida produce este código?

```
int suma, iteraciones;

suma = iteraciones = 0;

for (int i = 1; i <= 5; i++) {
  for (int j = 1; j <= 5; j++) {
    suma = suma + j;
    iteraciones ++;
  }
}
```

Solución

- Iteraciones 25. Suma $= 5 * (1 + 2 + 3 + 4 + 5) = 75$.
- En general:
  - iteraciones $= n^2$
  - suma $= n * \sum_{i=1}^{i=n} i = n\frac{n^2+n}{2} = \frac{n^3+n^2}{2}$

Anidamiento de Bucles

¿Qué salida produce este código?

```
1  int suma, iteraciones;
2  suma = iteraciones = 0;
3
4  for (int i = 1; i <= 5; i++) {
5    for (int j = i; j <= 5; j++) {
6      suma = suma + j;
7      iteraciones ++;
8    }
9  }
```

Anidamiento de Bucles

¿Qué salida produce este código?

```
1  int suma, iteraciones;
2  suma = iteraciones = 0;
3
4  for (int i = 1; i <= 5; i++) {
5    for (int j = i; j <= 5; j++) {
6      suma = suma + j;
7      iteraciones ++;
8    }
9  }
```

Solución

- Iteraciones $5 + 4 + 3 + 2 + 1 = 15$.
- Suma $= (1 + 2 + 3 + 4 + 5) + (2 + 3 + 4 + 5) + (3 + 4 + 5) + (4 + 5) + 5 = 5 * 5 + 4 * 4 + 3 * 3 + 2 * 2 + 1 * 1 = 55$

Anidamiento de Bucles

Solución

- Iteraciones $= \sum_{i=1}^{i=n} i = \frac{n(n+1)}{2}$
- Suma $= \sum_{i=1}^{i=n} i^2 = \frac{1}{6} n(n+1)(2n+1)$

Contenido del tema

1. Estructuras condicionales
2. Estructuras repetitivas
3. Tipo de dato enumerado

Tipo de Dato Enumerado

Situaciones en las que necesitamos manejar información que tiene un conjunto cerrado de valores posible.

- Calificación ECTS de un alumno: {A, B, C, E, F,G}
- Tipo de letra: {mayúscula, minuscula, otro}
- Posición podio: {oro, plata, bronce}
- Día de la semana: {lunes, . . . , domingo}
- Categoría labotal: {administrativo, programador, analista, directivo}

Con **char** o **int** propenso a errores. Con tipo **bool**, una variable por cada categoría.

Definición y uso

Definición y uso tipo enumerado

```
enum class CalificacionECTS {A, B, C, D, E, F, G};
enum class PuestoPodium {oro, plata, bronce};

int main() {
  CalificacionECTS calificacion_estudiante = CalificacionECTS::A;
  PuestoPodium puesto_deportista = PuestoPodium::plata;
  if (puesto_deportista == PuestoPodium::plata) {
    ....
  }
}
```

Desafío: Problema FizzBuzz

Problema FizzBuzz

Este es un test que se hizo popular hace unos años porque tanto Google como Microsoft lo usaron en sus primeras entrevistas de trabajo.

Curiosidades

- Lo fallaba más del 80% de los candidatos.
- Motivo de descarte.

Enunciado

"Escribe un programa que muestre los números del 1 al 100. Pero para múltiplos de tres muestra"Fizz" en vez del número, y para los múltiplos de cinco muesta "Buzz". Para los números que son múltiples tanto de tres como de cinco muestra "FizzBuzz".